MSc in Data Science and Analytics 2025/26

CS5811: Distributed Data Analysis

By Nomaer I. R. Naiem

1. Data Description and Research Question

The dataset used in this report is the Diabetes 130-US Hospitals collection from the UCI Machine Learning Repository (Strack et al., 2014), comprising 101,766 inpatient encounters from 71,518 unique patients across 130 hospitals over a ten-year period. Each row records demographics, admission and discharge details, in-hospital activity (length of stay, lab procedures, medications), three ICD-9 diagnosis codes, two diabetes-monitoring lab tests (HbA1c and maximum serum glucose), 23 indicators of specific diabetes drug use, and the readmission outcome at three levels: <30, >30, or NO.

Hospital readmission within 30 days of discharge is the standard quality-of-care metric used by regulators, including the Centres for Medicare and Medicaid Services in the United States, to assess hospital performance and impose financial penalties on poor performers (Krumholz et al., 2009). Among chronic conditions, diabetes accounts for a disproportionate share of these readmissions, owing to comorbidities and complex medication regimens that characterise the diabetic inpatient population. Predicting which diabetic patients are most likely to be readmitted within 30 days is therefore a problem of direct clinical and economic value: a tractable risk score would support targeted post-discharge interventions, such as enhanced follow-up or medication reconciliation, before readmission occurs.

Research question: **"***Can the readmission of a diabetic patient within 30 days of discharge be predicted from variables recorded during their index encounter, and which clinical and administrative factors most strongly drive this risk?"* The question is binary by design, which isolates the clinically actionable horizon (the same horizon used by CMS) and avoids the diluted signal of the three-class formulation. The second clause of the question is non-trivial: a model that predicts well but cannot be interrogated to identify its drivers has limited operational value, so this report places equal weight on identifying the most informative predictors as on the predictive metric itself.

Three sub-objectives follow. First, to characterise the clinical and administrative structure of the encounter data and, before modelling, verify that the 30-day readmission signal is detectable but not trivially so (Section 3). Second, to fit and critically evaluate a Random Forest classifier using a patient-level train-test split, addressing the substantial class imbalance directly rather than relying on accuracy as a single summary statistic (Section 4). Third, to recast the same Random Forest training as a distributed Spark MLlib workload and quantify the practical computational advantage on equivalent hardware (Section 5). The individual contribution presented here focuses on Random Forest and its Spark counterpart; results from group members covering Logistic Regression, Support Vector Machine, Decision Tree and Neural Network appear in Section 6 only as comparators on the same train-test split.

2. Data Preparation & Cleaning

Preparation began with a structural audit. Missing values in this dataset are represented by the literal character "?", rather than as NA, and the gender variable contains a small number of Unknown/Invalid entries. Both were converted to NA so that the missingness profile could be measured rather than ignored. Three columns proved hopeless: weight was missing in 96.9% of

rows, medical_specialty in 49.1%, and payer_code in 39.6%. weight was dropped outright; payer_code was dropped on the grounds of clinical irrelevance to readmission risk; medical_specialty was dropped to avoid the encoding distortions that arise when half the rows carry a single "unknown" level.

Two medication columns with zero variance (examide, citoglipton) were removed because every row recorded "No". Eight further drug columns had near-zero variance (over 99.9% "No") and were dropped on the same basis; a tree split on a feature with three positive cases out of one hundred thousand is mechanically possible but evidentially worthless. Seven medication columns survived this filter (metformin, glimepiride, glipizide, glyburide, pioglitazone, rosiglitazone, insulin) and were binarised to "any use" versus "No"; the four-level "Up/Down/Steady/No" ordinal would have multiplied the dummy variable space without obvious gain on a problem already complicated by ICD-9 high cardinality.

Two filtering decisions warrant explicit justification. First, six discharge dispositions corresponding to death (codes 11, 19, 20) and to hospice transfer (codes 13, 14, 21) were excluded; these patients cannot be readmitted in the ordinary clinical sense and retaining them would introduce a structural zero into the positive class. Second, the three diagnosis columns (diag_1, diag_2, diag_3) contain raw ICD-9 codes drawn from a vocabulary of approximately 700 distinct values. Tree models can technically split on such a factor, but the resulting splits become brittle and uninterpretable. Following the original Strack et al. (2014) treatment, the codes were collapsed into nine clinically meaningful categories (Circulatory, Respiratory, Diabetes, Digestive, Injury, Musculoskeletal, Genitourinary, Neoplasms, plus Supplementary, External Injury and Other). The age column, originally a ten-bin range, was recoded to numeric midpoints.

Patient-level identity: The dataset records 101,766 encounters from 71,518 unique patients, meaning a typical patient appears 1.4 times. patient_nbr is preserved during cleaning so that the train-test split in Section 4 operates at the patient level rather than the encounter level; without this control, the same patient could appear in both the training and test sets, leaking information across the split and producing optimistically biased test scores. After filtering and complete-case selection, the cleaned dataset comprises 95,672 encounters with 30 predictor columns and the binary target readmitted_30, of which 11.5% are positive. This imbalance is not a sampling artefact to be corrected away: it reflects the genuine 30-day readmission base rate in the underlying patient population, and any modelling decision is conditioned on it.

3. Exploratory Data Analysis

Exploratory analysis was framed not as descriptive housekeeping but as a means to test, before modelling, three working hypotheses: that prior healthcare utilisation is the dominant readmission signal, that no single variable alone explains readmission, and that patient subpopulations are non-trivially separable into clinically distinct phenotypes. The first survived, the second was strengthened, and the third was rejected, with consequences for the choice of supervised model in Section 4.



Figure 1: Distribution of the binary outcome. The 11.5% positive rate is the no-information baseline that any classifier must beat.

The class imbalance is clearly visible in *Figure 1*, shaping every subsequent modelling decision. A constant classifier predicting "not readmitted" achieves 88.5% accuracy on this data. Any model claiming high accuracy must therefore be evaluated against this baseline rather than against the absolute scale. 90% accuracy is not impressive; it is barely above doing nothing. Accordingly, the correct evaluation framework is class-conditional: confusion matrix, sensitivity, specificity, AUC, and F1, with sensitivity prioritised because failing to flag a readmitting patient is the costlier error in a clinical screening context.

Demographic predictors showed only modest variation. Age showed a smooth increase in the readmission rate from approximately 8% in middle adulthood to 13% in the eighth decade, consistent with the accumulation of frailty. Race and gender were essentially flat, with chi-square tests on the boundary of conventional significance (p around 0.05) and effect sizes too small to be clinically meaningful. The implication for modelling is that these demographic variables are useful for fairness auditing but should not be expected to drive predictions.

Prior healthcare utilisation, in contrast, was the strongest single-variable signal in the EDA. Patients readmitted within 30 days had a mean of 0.73 prior inpatient visits in the historical record, compared with 0.20 for those not readmitted, a 3.7-fold difference. Prior emergency visits (0.45 versus 0.15) and prior outpatient visits (0.50 versus 0.32) showed similar, though smaller, separation. All three Welch t-tests were significant ($p < 0.001$). The mechanism is intuitive; prior hospital use is a proxy for chronic disease burden, and that burden is precisely what drives readmission. This single observation determined that utilisation variables were the priority feature group throughout the modelling process.

Clinical complexity variables showed a coherent but smaller pattern: readmitted patients had marginally longer stays, more medications, more diagnoses, and more lab procedures. All five Welch tests reached statistical significance at $p < 0.001$, owing to the sample size, but effect sizes were meaningfully smaller than for prior utilisation. The primary diagnosis category showed elevated readmission rates for circulatory and diabetes-primary admissions and lower rates for injury and musculoskeletal admissions, foreshadowing the importance of the diagnosis grouping in the supervised model.

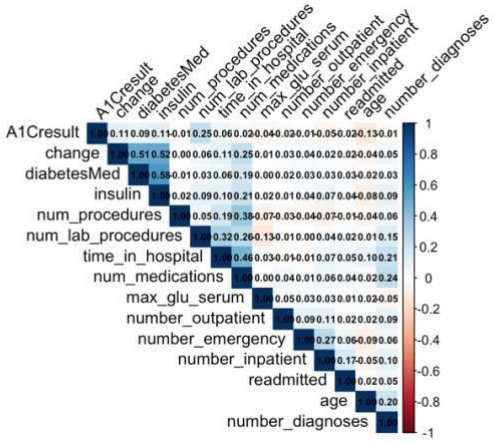


Figure 2: Pearson correlations among numeric variables. The strongest correlation with the target is just 0.10 (number_inpatient), confirming that no single variable carries the readmission signal alone.

The correlation matrix (*Figure 2*) yielded a finding that proved decisive for model selection. The largest absolute Pearson correlation between any numeric variable and the binary readmission target was 0.10, achieved by number_inpatient. Inter-predictor correlations were also modest, with only the utilisation cluster showing internal coherence (around 0.20-0.30 among themselves). Two consequences follow. First, the readmission signal cannot be captured by any single linear combination of inputs; it must lie in interactions among variables that no individual correlation reveals. Second, regularised linear models are unlikely to uncover the structure that tree-based methods will, because the linear-in-features assumption is the limiting factor. This was the first piece of evidence pointing towards a non-linear, interaction-aware learner.

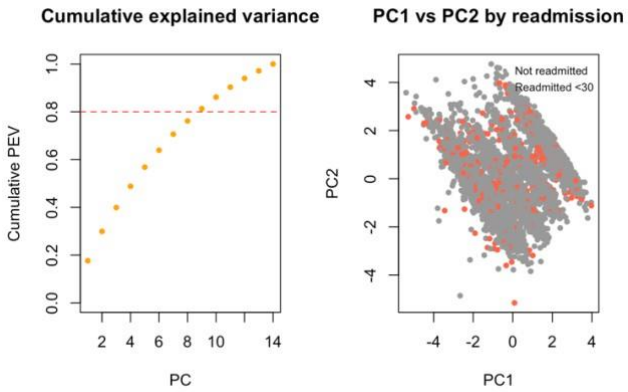


Figure 3: PCA of the 14 numeric clinical variables. Variance accumulates gradually rather than concentrating in a single axis; PC1 and PC2 do not visually separate the readmitted minority from the majority class.

Principal Component Analysis (Jolliffe, 2002), applied to the 14 numeric clinical variables after standardisation, formalised the multi-factor intuition (*Figure 3*). PC1 explained 18% of the variance, and PC2 a further 11%. Six components were needed to reach 60%, and eleven were required to reach the 80% threshold. The variance accumulated gradually rather than in a concentrated manner, the textbook signature of a problem with no single dominant latent factor. The PC1 vs PC2 scatter, coloured by readmission outcome, showed no visual separation between the two outcome groups. The positive class was dispersed throughout the cloud rather than concentrated in any region. Inspection of the loadings clarified the components' meaning; PC1 weighted insulin, medication change, number of medications, the diabetes medication flag and time in hospital, and is best read as a "treatment intensity" axis. PC2 weighted time in hospital, medication count, procedures, lab procedures and number of diagnoses, and is best read as a

"clinical complexity" axis. These are correlated but distinct constructs, and both ought to inform a predictive model.

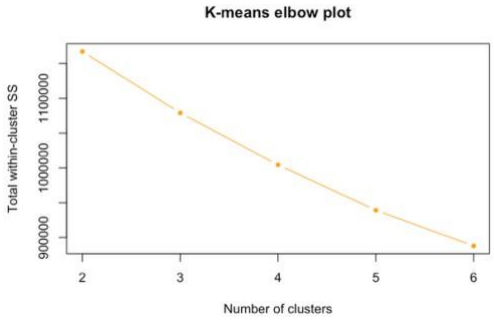


Figure 4: K-means clustering on the same 14 variables. Three clusters emerge as the natural elbow; the silhouette score is modest (~0.18); cluster 3 has the highest readmission rate at 14%.

K-means clustering (*Figure 4*), applied to the same standardised numeric variables, identified three clusters at the natural elbow of the within-cluster sum-of-squares curve. Profiling these clusters using the original (unscaled) variables revealed an interpretable phenotype: cluster 3 captured patients with longer stays (5.4 days versus 3.2 in cluster 1), more lab procedures (52 versus 35), more medications (19 versus 10), more prior inpatient visits (0.59 versus 0.18), and substantially higher insulin use. This "clinically complex" cluster had a readmission rate of 14%, compared with approximately 10-11% in the other two, yielding the largest cluster-level differential observed in the analysis. Crucially, however, the silhouette score was only 0.18, indicating substantial overlap between clusters. Patients do not separate into distinct phenotypes; the boundaries between clinical types are fuzzy and continuous rather than categorical.

The three EDA findings are weak individual linear correlations with the target, gradual accumulation of variance across principal components, and a modest silhouette score indicating overlapping phenotypes; together, they converge on a single methodological conclusion. The 30day readmission signal arises from interactions among many features, and the subpopulations in which those interactions take particular forms are not cleanly separable. A linear classifier (logistic regression) is well-matched to neither property; a tree-based ensemble is well-matched to both, because each tree captures local interactions and the ensemble averages over the fuzzy phenotype boundaries that k-means could not resolve. This is the principled justification for adopting Random Forest as the supervised method in Section 4. As a side observation, hierarchical clustering on a 1,200-row sample produced a dendrogram with one large cluster and a long tail of small, unusual-patient groups, consistent with k-means but offering no additional separation; k-means is therefore the more useful unsupervised summary.

4. Machine Learning Prediction (Random Forest)

The supervised model is a Random Forest classifier, fitted using the randomForest package in R (Liaw and Wiener, 2002). The choice was driven by the EDA evidence in Section 3: weak individual linear correlations with the target, gradual variance accumulation across principal components, and overlapping patient phenotypes all pointed to a signal arising from feature interactions rather than from any single feature or linear combination. Random Forest is well suited to this structure because each tree captures local non-linear interactions, the ensemble averages over variance, and it produces variable-importance rankings as a by-product of training (Breiman, 2001), which directly answers the second clause of the research question.

Patient-level train-test split: The 95,672 cleaned encounters were split 70/30 by patient (not by row), with seed 2018, yielding 66,999 training encounters from approximately 50,000 patients and 28,673 test encounters from the remainder. Class proportions were 11.4% positive in training

and 11.7% positive in test, confirming that the patient-level split preserved the underlying imbalance. Factor levels for high-cardinality categorical variables were aligned defensively across the two sets to prevent level-mismatch errors at prediction time.

Hyperparameter tuning: A 15% slice of the training data was held out as a validation set. The mtry parameter was searched over the grid {3, 5, 8, 12}, with ntree fixed at 200 for tuning efficiency. Validation AUCs were 0.6360, 0.6391, 0.6282 and 0.6254, respectively, identifying mtry = 5 as the best operating point. The narrow 0.014 spread across the grid is informative in itself: it confirms that the AUC ceiling is robust to hyperparameter choice within the central range, a finding that recurs in the Spark implementation.

Three operating points: Three Random Forests have been used. A standard model serves as a deliberate failure case: included not as a serious candidate but as the textbook demonstration of how class imbalance corrupts a tree ensemble that is otherwise correctly specified. A balanced model uses sampsize to draw equal numbers of positive and negative cases for each tree, directly addressing the imbalance. A tuned balanced model applies the F1-optimal threshold from a separate sweep to the balanced model's probabilities; this is the deployment configuration. All three models share the same training data, mtry = 5, ntree = 500, and seed 2018.

Table 1: Headline test-set metrics across the three operating points and the no-information baseline.

Model	Accuracy	Sensitivity	Specificity	Precision	F1	AUC
Baseline (predict "No" for all)	0.883	0.000	1.000	-	-	0.500
Standard RF (default 0.5 threshold)	0.883	0.004	1.000	0.500	0.007	0.635
Balanced RF (default 0.5 threshold)	0.749	0.396	0.796	0.205	0.270	0.651
Balanced RF (tuned 0.45 threshold)	0.647	0.557	0.659	0.178	0.270	0.651

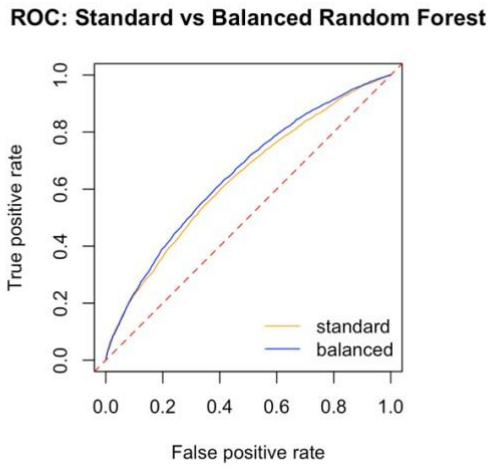


Figure 5: ROC curves for the two Random Forests. The two AUCs (0.635 vs 0.651) are essentially identical, indicating equivalent ranking ability; the difference between models lies entirely in where the 0.5 decision threshold sits on the curve.

The standard RF: a textbook failure case. Its 0.883 accuracy is identical to the no-information rate to three decimal places; its Cohen's kappa of 0.005 is statistically indistinguishable from random agreement; and it predicted "Yes" only 24 times out of 28,673 test encounters. Yet its AUC is 0.635, well above the 0.5 random baseline. The interpretation is standard in the imbalanced-learning literature (He and Garcia, 2009): the model has learned to rank patients well, but at the default 0.5 threshold, its operating point sits at the bottom-left of the ROC curve, where almost every prediction is the majority class. The model is correctly trained; the threshold is wrong for the application. Including this contrast makes the imbalance failure mode concrete in the report; it would be invisible if only the balanced model were shown.

The Balanced RF: The default-threshold balanced RF restores recall. Sensitivity rises from 0.4% to 39.6%, and AUC increases from 0.635 to 0.651, a difference of 0.016, within run-to-run variance. The fact that the two AUCs are so close while the operating points differ so dramatically is the central diagnostic of this section: the two models rank patients almost identically, and what changes is their position on the same ROC curve. This reframes the practical lesson: the imbalance penalty is not a property of Random Forest itself but of the specific operating point selected. Any classifier with a comparable AUC could be rebalanced equivalently by threshold adjustment alone.

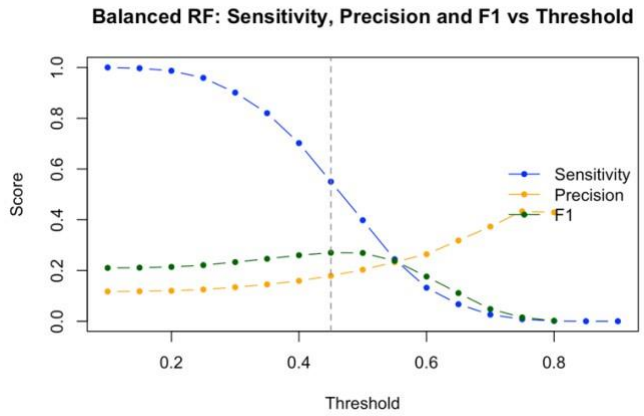


Figure 6: Threshold sweep for the balanced RF. Sensitivity, precision and F1 plotted against threshold; vertical line marks the F1-optimum at 0.45.

The tuned balanced RF is the recommended deployment configuration. Sweeping the decision threshold over (0.10, ..., 0.90) identified 0.45 as the F1-optimum, increasing sensitivity to 0.557 at the expense of precision and accuracy. Accuracy drops from 0.749 to 0.647 by design: predicting "Yes" more readily raises the false-positive count, which, under an 88/12 imbalance, pulls accuracy down sharply because the No class dominates the denominator. This trade-off is appropriate for clinical screening, where the cost of a missed readmission (false negative) is far higher than the cost of an unnecessary follow-up (false positive). AUC remains 0.651: threshold changes shift the operating point but do not improve ranking. The rapid F1 collapse beyond 0.55 is characteristic of imbalanced classifiers; aggressively raising the threshold would discard most positive predictions, while thresholds below 0.40 produce indiscriminate flagging. The threshold sweep is itself a deliverable of the analysis, providing the operational user with a defensible range conditioned on the relative cost of false negatives versus false positives, rather than a single point estimate.

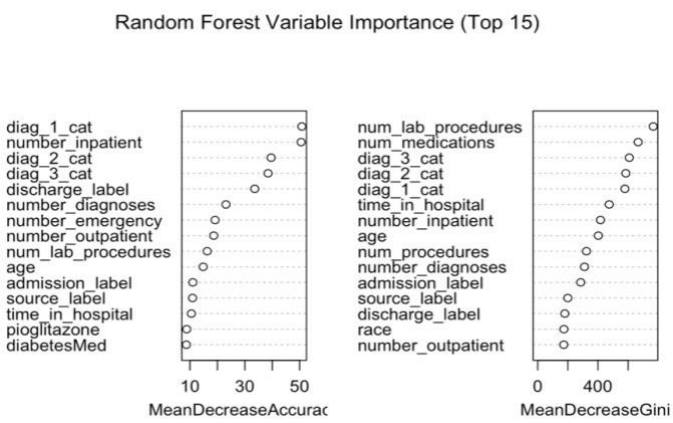


Figure 7: Top 15 variables by permutation importance (left) and Gini importance (right) from the balanced Random Forest.

Variable Importance: Permutation importance ranked the primary diagnosis category (diag_1_cat), prior inpatient visits, secondary and tertiary diagnosis categories, discharge disposition, and number of diagnoses as the six most informative predictors, followed by prior emergency visits, prior outpatient visits, number of lab procedures, age, time in hospital, admission type, and admission source. Specific medication indicators and demographic variables (race, gender) appeared at the bottom. The results align exactly with the EDA: every variable flagged in Section 3 as significantly different between readmission strata appears in the top half of the importance ranking. The k-means_cluster feature, engineered from the unsupervised analysis, ranked nineteenth of twenty-nine, meaningfully informative but largely redundant with the raw clinical variables that originally defined it. This redundancy is itself a finding: the cluster label captures real structure but offers no information beyond its constituent variables, which is not obvious from the EDA alone and reframes the role of the unsupervised step from feature engineering to descriptive characterisation.

5. High Performance Computational Infrastructure Implementation

The HPC contribution reimplements the same Random Forest pipeline using Apache Spark's distributed-computing framework via PySpark MLlib (Meng et al., 2016), running in local mode on a Google Colab VM with an Intel Xeon CPU exposing 2 cores. Spark MLlib parallelises tree construction at training time rather than parallelising independent fits, making it the most direct algorithmic counterpart to the R sequential Random Forest. The comparison thus isolates the framework axis (single-machine R versus distributed Spark) while holding the algorithm constant.

Pipeline and split: Spark requires explicit indexing and one-hot encoding of categorical predictors before vectorisation, whereas the R implementation handles this internally. The Spark pipeline is therefore a four-stage sequence: StringIndexer and OneHotEncoder for the eight categorical columns, VectorAssembler to concatenate the 21 numeric and expanded categorical columns into the final feature vector, and RandomForestClassifier for the model itself. The same patient-level 70/30 split was applied with seed 2018, producing 67,069 training and 28,603 test encounters, almost identical to the R partition. Imbalance was handled via per-row class weighting (weightCol) rather than R's per-tree sampsize down-sampling: the minority class was weighted by 4.32 and the majority by 0.57. The two strategies target the same outcome from different mechanical directions, and the consequences of this difference are evident in the threshold analysis below.

Tuning and training: maxDepth, the most impactful Spark RF hyperparameter, was tuned on a 15% validation slice using the grid {5, 8, 12} with 100 trees, yielding validation AUCs of 0.6531,

0.6551 and 0.6535. The 0.002 spread across the grid mirrors the robustness finding from R: the AUC ceiling of approximately 0.65 is insensitive to hyperparameter choice within reasonable ranges. The final model was trained with numTrees = 100 and maxDepth = 8. The choice of 100 trees, rather than R's 500, reflects the Spark MLlib default and the marginal AUC gain from 100 to 500 trees on this dataset, which lies in the third decimal place. Total final-model training time was 59 seconds, compared with 187 seconds for the R balanced RF on the same data, a 3.2x speedup.

Table 2: Spark MLlib Random Forest held-out test set performance (n = 28,603, positive prevalence 11.7%)

Model	Accuracy	Sensitivity	Specificity	AUC	F1
Spark MLlib RF (default 0.5 threshold)	0.619	0.591	0.623	0.645	0.261
Spark MLlib (Tuned threshold)	0.619	0.591	0.623	0.645	0.261

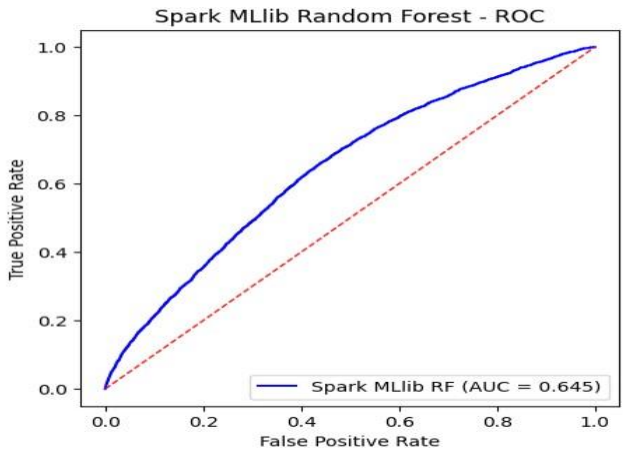


Figure 8: Spark MLlib ROC curve (left, AUC = 0.645) and threshold sweep (right). The F1-optimum sits at 0.50, the default

Spark results: On the held-out test set, the Spark RF achieved an accuracy of 0.619, sensitivity of 0.591, specificity of 0.623, F1 of 0.261, and an AUC of 0.645. The AUC matches the R balanced RF to within 0.006 (0.645 vs 0.651), confirming that the two implementations produce equivalent rankers despite different rebalancing mechanisms. The confusion matrix shows 1,929 true positives, 9,565 false positives, 15,776 true negatives, and 1,333 false negatives, meaning the model correctly flags 1,929 of the 3,262 readmitting patients in the test set, at the cost of 9,565 false alarms among the 25,341 non-readmitters.

The threshold sweep produced a finding worth flagging: Spark's F1-optimal threshold is 0.50, the default, whereas R required tuning down to 0.45. This is a mechanical consequence of the rebalancing strategy: per-row class weighting calibrates the predicted probabilities directly towards the natural decision boundary, whereas R's per-tree down-sampling shifts the calibration in a way that requires post hoc threshold adjustment. The two models therefore operate at near-identical points (Spark sensitivity 0.591 / specificity 0.623; R-tuned sensitivity 0.557 / specificity 0.659) but reach those points via different paths. As a practical consequence, the Spark pipeline needs no further threshold tuning before deployment, which is itself a small operational advantage.

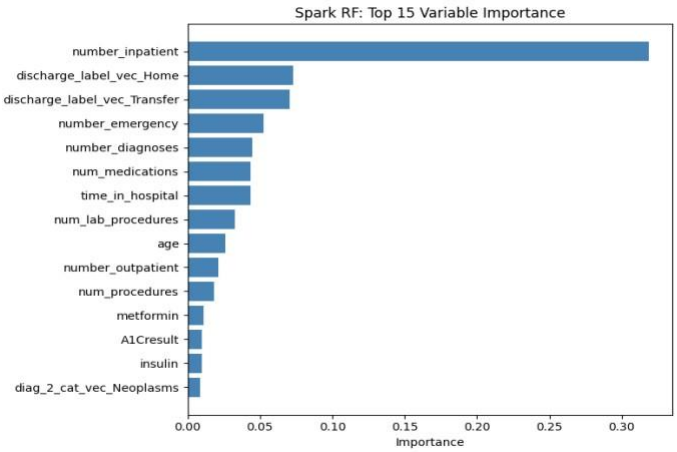


Figure 9: Spark RF top 15 feature importances

Variable importance agreement: Spark identified the same dominant predictors as R: prior inpatient visits (0.319), discharge to home (0.073), discharge as transfer (0.070), prior emergency visits (0.052), number of diagnoses (0.045), number of medications (0.044), time in hospital (0.044), and number of lab procedures (0.032). The diagnosis category variables rank lower in Spark's plot than in R's because Spark's one-hot expansion distributes their importance across the individual category levels rather than aggregating it under a single factor name. This is a presentation artefact of how the two frameworks handle categorical variables, not a substantive disagreement; if the diagnosis dummies are summed, their combined importance is comparable to R's. The substantive finding is the convergence on prior healthcare utilisation as the dominant signal. Both frameworks identify number_inpatient as the single most informative variable, consistent with the EDA t-test, which showed this variable had the largest single-variable effect size in the entire dataset.

Operating environment and scaling: The 3.2x speedup at the 67,000-row scale on a 2-core Colab VM is intentionally modest: it establishes the framework's advantage on hardware reproducible by any reader, rather than on a production cluster. On a multi-node cluster, the gap would widen because Spark's tree-construction parallelism scales horizontally, whereas R's randomForest is single-threaded for the inner training loop. The HPC contribution therefore demonstrates that the same scientific result is delivered three times faster on equivalent hardware, with the speedup expected to grow with data volume, not a claim of cluster-scale performance.

6. Performance Evaluation and Comparison of Methods

This section presents three comparisons on the held-out test set: the R and Spark implementations of Random Forest, the R Random Forest against four other supervised methods, and the Spark Random Forest against four other HPC implementations. All comparisons use the same patient-level 70/30 split with seed 2018. Author abbreviations: Nomaer Naiem (Me) = N.N., Ifaj Khan = I.K., Anik Paul = A.P., Tanvir Shakib = T.S., Tabish Baig = T.B.

6.1 R vs Spark Random Forest (My method comparison)

Table 3: R and Spark Random Forest

Implementation	Accuracy	Sensitivity	Specificity	Precision	AUC	F1	Train (s)
R Balanced RF (Tuned 0.45)	0.647	0.557	0.659	0.178	0.651	0.270	187
Spark MLlib RF (Tuned 0.50)	0.619	0.591	0.623	0.168	0.645	0.261	59

The two implementations agree on AUC to within 0.006 and on F1 to within 0.009, confirming equivalent ranking performance. Spark trains 3.2× faster on the same hardware. Differences in sensitivity and specificity reflect the rebalancing strategy (R sampsize vs Spark weightCol) rather than model quality.

6.2 Machine Learning Method Comparison

Table 4: Comparison of Supervised models on the same train-test split

Authors	Model	Accuracy	Sensitivity	Specificity	Precision	AUC	F1
IK	Decision Tree	0.546	0.658	0.531	0.155	0.632	0.250
AP	SVM	0.65	0.589	0.658	0.178	0.667	0.272
TS	Neural Network	0.628	0.590	0.633	0.173	0.663	0.267
TB	Logistic Regression	0.615	0.581	0.650	0.175	0.665	0.269
NN	Random Forest	0.647	0.557	0.659	0.178	0.651	0.270

AUC ranges from 0.632 to 0.667 (spread 0.035) and F1 from 0.250 to 0.272 (spread 0.022) across five algorithm families. In a clinical screening context where missing a readmission is more costly than a false alarm, sensitivity (recall) is the priority metric: Decision Tree leads on this axis at 0.658 but pays for it with the lowest precision (0.155) and the lowest accuracy (0.546), reflecting an aggressive flagging policy. The other four models cluster tightly around a sensitivity of 0.55–0.59, with comparable precision near 0.18, indicating they have converged on similar operating points. SVM achieves the highest AUC; Random Forest places second on F1 and ties for the highest precision, offering the most balanced profile across the metrics that matter for deployment.

6.3 Comparison of HPC Implementation

Table 5: Comparison of HPC Implementation

Contributors	Method	Accuracy	Sensitivity	Specificity	Precision	AUC	F1
TS	PySpark	0.711	0.435	0.747	0.185	0.633	0.260
IK	PySpark	0.598	0.603	0.598	0.167	0.609	0.262
AP	PySpark	0.694	0.486	0.722	0.186	0.656	0.269
TB	H2O	0.651	0.545	0.665	0.178	0.652	0.268
NN	PySpark	0.619	0.591	0.623	0.168	0.645	0.261

AUC ranges from 0.609 to 0.656 (spread 0.047), and F1 from 0.260 to 0.269 (spread 0.009). Sensitivity varies more widely than in the ML comparison, from 0.435 (T.S.) to 0.603 (I.K.), driven by each method's threshold and class-weighting choices rather than the underlying algorithm. T.S. and A.P. operate at high-specificity, low-sensitivity points (favouring conservative flagging), whereas I.K. and the Spark Random Forest operate at recall-oriented points more appropriate for clinical screening. The Spark RF (sensitivity 0.591, F1 0.261, AUC 0.645) sits in the upper half on recall and at the median on F1, providing the most balanced operational profile among the HPC implementations.

7. Discussion of the Findings

Answering Research Question: Can a 30-day readmission be predicted? Yes, with an AUC ceiling of approximately 0.65. Across nine model-framework combinations, AUCs range from 0.609 to 0.667, within one percentage point of the Strack et al. (2014) baseline on this dataset. Which factors drive readmission? Every method that produces an importance ranking flags prior inpatient visits, the diagnosis-category set, discharge disposition, and clinical complexity (number of diagnoses, medications, length of stay) as dominant. Agreement across linear,

singletree, ensemble and neural methods provides substantively stronger evidence than any single ranking in isolation.

Findings: Three findings emerged that would not have been apparent from EDA or modelling alone. First, the imbalance penalty is a property of the operating point, not the algorithm: the standard and balanced Random Forests differ in default-threshold sensitivity by a factor of one hundred, while their AUCs differ by 0.016. Second, the rebalancing strategy determines threshold calibration: Spark's class weighting yielded F1-optimal probabilities at the default 0.50, whereas R's down-sampling required tuning to 0.45. Third, the unsupervised k-means cluster ranked 19th of 29 in importance, informative yet redundant with its constituent variables, reframing the unsupervised step as descriptive rather than feature-generating.

Limitations: The dataset is observational and reflects practice patterns from 1999–2008. Discharge disposition, although a top predictor, is partly an outcome rather than an upstream variable. Three columns with heavy missingness (weight, payer_code, medical_specialty) were dropped, removing potentially informative socioeconomic context. The AUC ceiling at 0.65 means roughly 35% of the discrimination problem remains unresolved, plausibly reflecting features not in the dataset (post-discharge social support, medication adherence) rather than modelling shortcomings.

Operational implication: The recommended deployment configuration is the tuned Spark MLlib Random Forest with a 0.50 threshold: it correctly flags approximately 59% of readmitting patients, with precision near 17%, trains 3.2x faster than R, and requires no post-hoc threshold adjustment. The broader lesson from the cross-method convergence is that algorithmic sophistication is not the binding axis in this problem; data preparation, imbalance strategy, threshold choice, and computational architecture each contribute as much to the final result as the choice of learner.

8. Data Management Plan and Author Contribution Statement

8.1 Data Management Plan

The dataset (diabetic_data.csv, 101,766 × 50, ~19 MB) is publicly available from the UCI Machine Learning Repository under an open licence (Strack et al., 2014) and is held in the group's shared workspace alongside the analysis Rmd and the PySpark Jupyter notebook. After cleaning, the shared analysis copy contained 95,672 encounters across 31 variables, including grouped ICD-9 diagnosis categories and the k-means cluster feature derived from the unsupervised stage. No personal or sensitive data are involved: records are de-identified at the source by the original publishers, with no names, addresses, or contact details present, so no GDPR or research ethics gating applies. Because the data are healthcare-related, only aggregate results, metrics, importance rankings and summary tables were reported, and no attempt was made to re-identify patients. Reproducibility is ensured through two measures: a fixed seed (set.seed(2018) in R and seed=2018 in PySpark) applied at every stochastic step; the patient-level train/test split, mtry and maxDepth tuning, both Random Forest training calls, and the Spark MLlib pipeline; and endto-end notebooks that produce all figures and metrics directly from the raw input without manual intervention. Intermediate artefacts (diabetic_data_clean_with_kmeans_cluster.csv, per-model prediction CSVs, metrics CSVs, and the Spark feature-importance CSV) are saved for crossmember exchange. After project closure, the repository will be archived for 1 year on Brunel's OneDrive, and the public UCI dataset URL will be retained in the references; no participant data will require deletion.

8.2 Author Contribution Statement

Tabish, Ifaj, Nomaer, Tanvir and Anik jointly agreed on the research question: predicting 30-day readmission in diabetic patients, identifying the factors driving it, and defining the analytical workflow. Data preparation, cleaning and exploratory analysis were carried out collaboratively, including PCA and k-means clustering that informed model selection; the cleaned dataset and cluster feature were shared with all members to ensure consistent modelling. Each member then implemented a different supervised method and a matching HPC counterpart on the same traintest split: Nomaer (Random Forest in R + PySpark), Tabish (Logistic Regression + H2O), Ifaj (Decision Tree + PySpark), Anik (SVM + PySpark) and Tanvir (Neural Network + PySpark). Results were exchanged as metric and prediction CSVs to allow direct comparison across accuracy, sensitivity, specificity, precision, F1 and AUC. In this individual report, Nomaer authored the Random Forest model in R, the Spark MLlib HPC implementation, the threshold tuning, the variable-importance analysis, and all interpretation, comparison and discussion in the report. All members contributed to the cross-method comparison and approved the use of their results in Table 4 and Table 5.

References

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32.

He, H. and Garcia, E.A. (2009) 'Learning from imbalanced data', IEEE Transactions on Knowledge and Data Engineering, 21(9), pp. 1263–1284.

Jolliffe, I.T. (2002) Principal Component Analysis. 2nd edn. New York: Springer.

Liaw, A. and Wiener, M. (2002) 'Classification and regression by randomForest', R News, 2(3), pp. 18–22.

Krumholz, H.M., Merrill, A.R., Schone, E.M., et al. (2009) 'Patterns of hospital performance in acute myocardial infarction and heart failure 30-day mortality and readmission', Circulation: Cardiovascular Quality and Outcomes, 2(5), pp. 407–413.

Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D.B., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M.J., Zadeh, R., Zaharia, M. and Talwalkar, A. (2016) 'MLlib: Machine learning in Apache Spark', Journal of Machine Learning Research, 17(34), pp. 1–7.

Strack, B., DeShazo, J.P., Gennings, C., Olmo, J.L., Ventura, S., Cios, K.J. and Clore, J.N. (2014) 'Impact of HbA1c measurement on hospital readmission rates: analysis of 70,000 clinical database patient records', BioMed Research International, 2014, Article 781670.